

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture No. 8



Re Cap Deterministic Finite Automaton (DFA)

Formal Definition

A *deterministic finite automaton* is a 5-tuple

$M = (K, \Sigma, \delta, s, F)$, where

K is a finite set, whose elements are called *states*

Σ is a finite set, called the *alphabet* set

$s \in K$ is the *initial state*

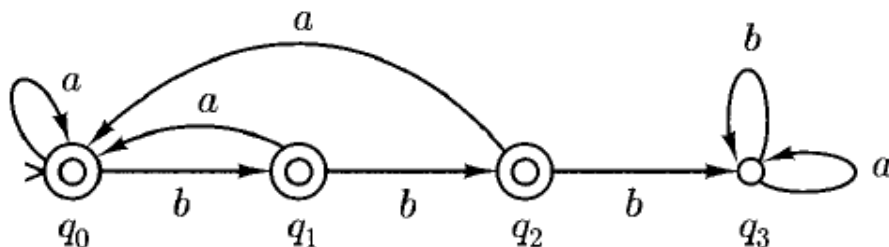
$F \subseteq K$ is set of *final/accept* states

$\delta : K \times \Sigma \rightarrow K$ is a function called the *transition function*

Re Cap - DFA - Example

Example:

$L = \{w \in \{a, b\}^* : w \text{ does not contain three consecutive } b\text{'s}\}$



and δ is given by the following table.

We let $M = (K, \Sigma, \delta, s, F)$, where

$$K = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\},$$

$$s = q_0,$$

$$F = \{q_0, q_1, q_2\},$$

q	σ	$\delta(q, \sigma)$
q_0	a	q_0
q_0	b	q_1
q_1	a	q_0
q_1	b	q_2
q_2	a	q_0
q_2	b	q_3
q_3	a	q_3
q_3	b	q_3

Computation

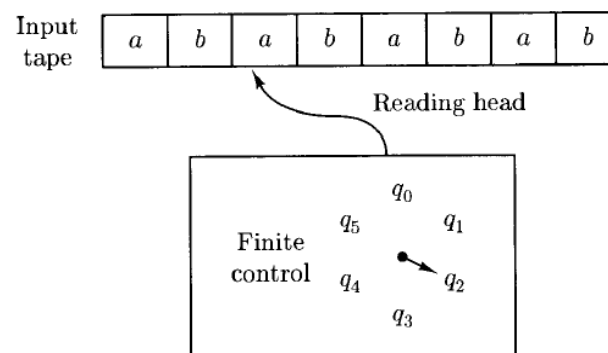
Configuration of DFA $(K, \Sigma, \delta, s, F)$ is an element of $K \times \Sigma^*$

In this figure the *configuration* is $(q_2, ababab)$

Computation is DFA moving

from a configuration to another configuration in one step.

Computation is an binary relation on two configurations of the DFA



Computation

Two *configurations* of a DFA are *related* if and only if one of the configuration can be *reached* from the other in one step.

If (q, w) & (q', w') are two configurations of DFA M , then $(q, w) \vdash_M (q', w')$

if and only if

$w = aw'$ for some symbol $a \in \Sigma$ and $\delta(q, a) = q'$

We say that (q, w) *yields* (q', w') in *one step*.

When the context is clear we drop M from $\vdash_M$

A *configuration* of the form (q, e) signifies that DFA M has read all its input & computation ceases.

Language accepted by DFA

We denote the *reflexive, transitive closure* of $\vdash_M$ by $\vdash_M^*$

So $(q, w) \vdash_M^* (q', w')$

means (q, w) *yields* (q', w') in *finitely* many (possibly 0) steps.

A string $w \in \Sigma^*$ is said to be *accepted* by DFA M if and only if there is a state $q \in F$ (set of final states) such that $(s, w) \vdash_M^* (q, e)$ where s is the initial state.

The *language accepted* by the DFA M , $L(M)$ is the set of all strings accepted by M .

Acceptance by DFA

Example

If DFA M is given and the input $babb$, then

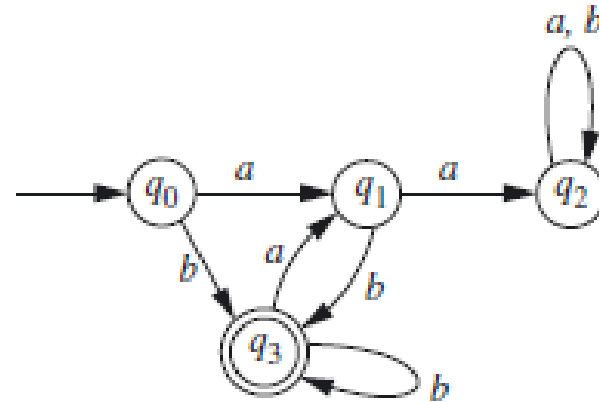
$(q_0, babb) \vdash_M (q_3, abb)$

$\vdash_M (q_1, bb)$

$\vdash_M (q_3, b)$

$\vdash_M (q_3, e)$

So $(q_0, babb) \vdash_M^* (q_3, e)$ and so $babb$ is *accepted* by M



Questions regarding languages accepted by DFA

Natural Questions:

1. Is languages accepted by DFA closed under *Union/Intersection/Complement?*
2. Is languages accepted by DFA closed under *Concatenation/Kleene Star?*
3. *How many* languages are accepted by DFA's?
4. Which language is not accepted by any DFA?

Union of Languages

Theorem: Languages accepted by DFA is closed under union. (*over the same alphabet Σ*)

Proof:

Let L & L' be two languages accepted by DFA M_1 & M_2 respectively.

To prove that $L \cup L'$ is also accepted by some DFA M .

In other words, M must have the property that for every string $w \in \Sigma^*$,

M accepts $w \iff M_1$ accepts w or M_2 accepts w

Idea: Is it possible to first check for M_1 & if it is unsuccessful then check for M_2 ?

This **idea does not work**, because *DFA M can read the input string w only once.*

Union of Languages

Correct approach:

To run M_1 and M_2 *simultaneously*.

How to achieve that?

Use Cartesian product

Let $M_1 = (K_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (K_2, \Sigma, \delta_2, s_2, F_2)$

If M is in state (r_1, r_2) , this means that

- if M_1 would have read the input string up to this point, then it would be in state r_1 , and
- if M_2 would have read the input string up to this point, then it would be in state r_2

Union of Languages

This leads to the **DFA** $M = (K, \Sigma, \delta, s, F)$, where

The set of **states** $K = K_1 \times K_2$ **Note:** Since K_1 & K_2 are **finite** so is $K_1 \times K_2$
 $\Sigma = \Sigma$

The **start state** $s = (s_1, s_2)$

The **transition function** $\delta : K \times \Sigma \rightarrow K$ given by

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

The set F of **accept states** of M is given by

$$F = \{(r_1, r_2) : r_1 \in F_1 \text{ or } r_2 \in F_2\}$$

To **finish** the proof, we have to show that this M accepts the language **$L \cup L'$**

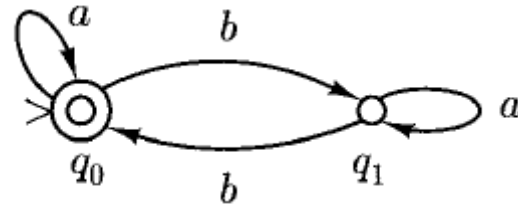
$$= (F_1 \times K_2) \cup (F_2 \times K_1)$$

Intuitively, it is clear from the discussion above. We leave the details.

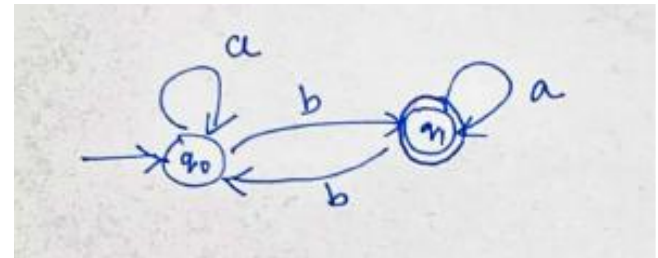
Complement of Language

What about complement?

L be the Strings in $\{a, b\}^*$ that have *even number of b 's*



L^c be the Strings in $\{a, b\}^*$ that have *odd number of b 's*



Easy:

Just swap the final & the non final states.

Intuitively, it is clear from the discussion above. We leave the details.

Questions regarding Languages accepted by DFA

Theorem:

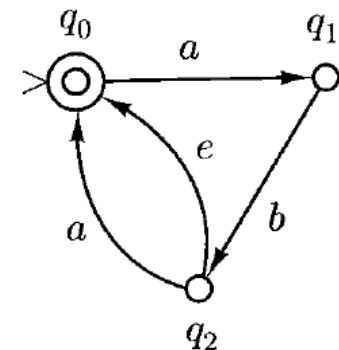
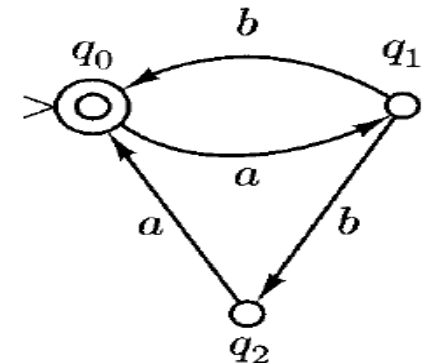
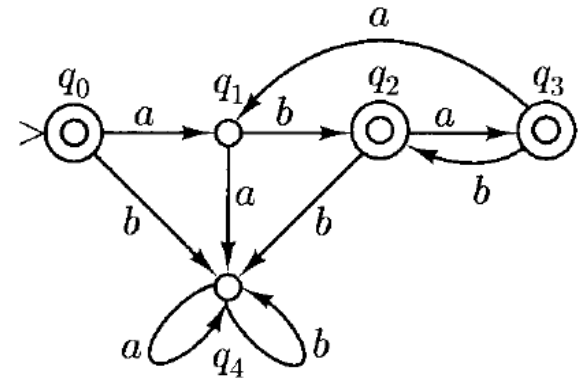
- Languages accepted by DFA is closed under union. (*over the same alphabet Σ*)
- Languages accepted by DFA is closed under complement.

Nondeterministic Finite State Automaton (NFA)

Consider $L = (ab \cup aba)^*$

We will *relax* the following *constraints* of DFA:

1. We *may not define* transition function for each pair of state, input symbol combination.
2. We shall allow *several possible 'next states'* for a given combination of current state & input symbol
3. The machine allows *"empty transitions"* i.e. machine can move from one state to another state without reading the input



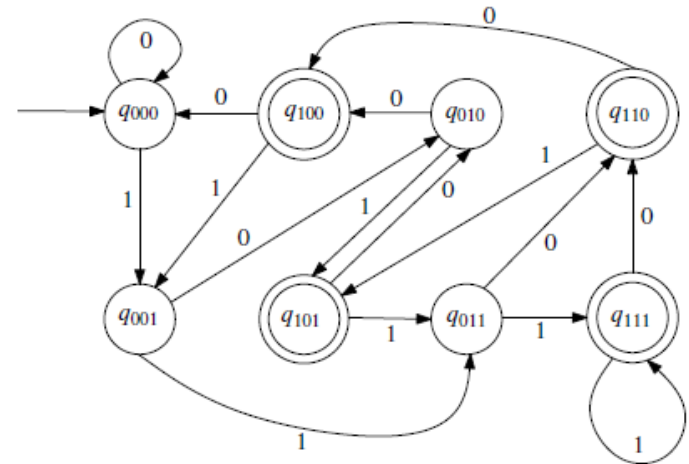
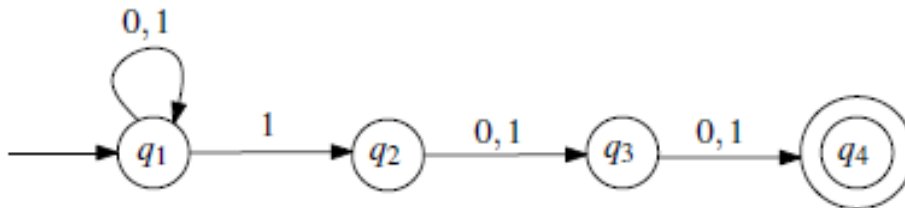
NFA - Example

Example: $L = \{w : \{0,1\}^* : w \text{ has a } 1 \text{ in the third position from the right}\}$

Idea

Strings of the type $x100$, $x101$, $x110$, $x111$ where $x \in \{0,1\}^*$ belongs to L

Remain in a state and when the input symbol is 1, *guess* that it is a string of the type above & then check for acceptance.



NFA

The automaton as it reads the input symbol, *may choose* at each step to go to any one of these *legal next states*.

The *choice* is *not determined by anything in our model* & is therefore said to be *nondeterministic*.

We think as NFA making the *right guess* for string acceptance.

Note:

NFA are *not realistic* model of computers.

Useful for *simplifying representation* of DFA

Important

Every NFA is *equivalent* to DFA.

Definition of NFA

Formal Definition

A *nondeterministic finite automaton* is a 5-tuple

$M = (K, \Sigma, \Delta, s, F)$, where

K is a finite set, whose elements are called *states*

Σ is a finite set, called the *alphabet* set

$s \in K$ is the *initial state*

$F \subseteq K$ is set of *final/accept* states

Δ , the transition relation is a subset of

$$K \times (\Sigma \cup \{e\}) \times K$$

NFA - Definitions

A triple of the form $(q, a, p) \in \Delta$, $a \in \Sigma \cup \{e\}$ is called a *transition* of M

This corresponds to machine is in state q and *the reading head* is pointing to a on input tape and moves to state p .

Configuration of NFA $(K, \Sigma, \Delta, s, F)$ is an element of $K \times \Sigma^*$

If (q, w) & (q', w') are two configurations of NFA M , then

$(q, w) \vdash_M (q', w')$ if and only if $w = aw'$ for some symbol $a \in \Sigma \cup \{e\}$ and $(q, a, q') \in \Delta$

Note that as in the case of DFA, $\vdash_M$ may not be *function*

As before, we denote the *reflexive, transitive closure* of

$\vdash_M$ by $\vdash_M^*$

NFA - Definitions

A string $w \in \Sigma^*$ is said to be *accepted* by NFA M if and only if there is a state $q \in F$ (set of final states) such that $(s, w) \vdash_M^* (q, e)$ where s is the initial state.

Note:

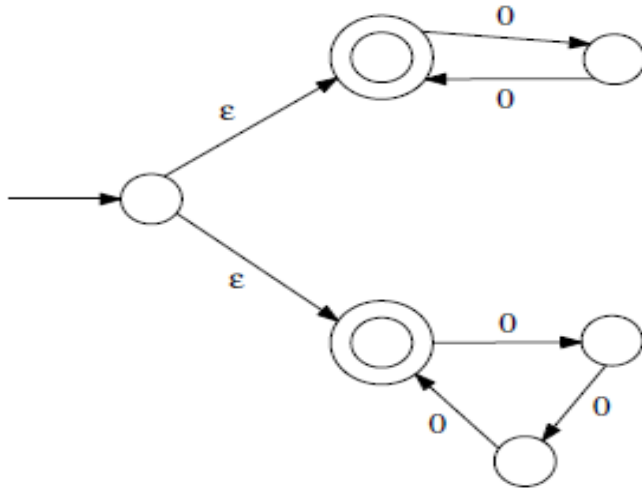
String is *accepted* if there is *some sequence of moves* that leads NFA from an initial configuration to a configuration in which the state component is a final state.

The *language accepted* by the NFA M , $L(M)$ is the set of all strings accepted by M .

Note: A string is *rejected* by M if there is *no sequence of moves* that leads NFA from an initial configuration to a configuration in which the state component is a final state.

NFA - Example

Example: Consider the NFA M



Note Any string not in L is **rejected** by M.
So 0^5 is always rejected by M

This NFA *accepts* the language

$$L = \{0^k : k \equiv 0 \bmod 2 \text{ or } k \equiv 0 \bmod 3\}$$

M *guesses* if the number of 0's is divisible by 2 or by 3
& *decides* which branch to take.